Prem Ravindra Jadhav

Pune, MH, IN | +91 7620349964 | premjadhav00002@gmail.com
LinkedIn | GitHub | Portfolio

Skills

DevOps & Cloud: Docker, Kubernetes, CI/CD (GitHub Actions), Nginx, Git, Linux (Bash/Zsh), Containerization, Redis, AWS (Basics)
Languages: Python (Scripting, Automation), Java (Spring), SQL, JavaScript (ES6+), C++
Backend Development: Spring Boot, Fast API, Flask, RESTful APIs, Express.js, Microservices Architecture, Hibernate, Spring Data JPA
Frontend & Tools: React.js, HTML5, CSS3, Postman, IntelliJ IDEA, VS Code, Linux CLI, System Design

Education

JSPM's Rajashri Shahu College of Engineering, Pune **Nov 2024 – 2027**
Bachelor of Technology in Computer Science **CGPA: 9.21**

Government Polytechnic, Nashik **Aug 2021 – Aug 2024**
Diploma in Computer Technology **91.18%**

Experience

MERN Stack Intern — SUMAGO Infotech, Nashik **Feb 2024 – Sept 2024**

- Built full-stack applications using React and Node.js; optimized API latency for production services.
- Developed Python scripts for internal data migration and automated testing of RESTful endpoints.
- Collaborated on deployment workflows, ensuring containerized environments remained consistent across teams.

Linux and Ethical Hacking Trainee — WhiteBand Associates **Mar 2024 – Aug 2024**

- Configured Linux environments and automated system tasks using Bash and Python scripting.
- Performed vulnerability assessments and system hardening for secure DevOps pipeline integration.
- Utilized networking tools to monitor traffic and identify security bottlenecks in server setups.

Projects

PreURL — High-Performance URL Shortener (Spring Boot & DevOps) **Dec 2025 – Present**

- Developed a scalable URL service with **Redis caching**, reducing database read operations by 70%.
- Implemented **Docker** containerization for multi-service orchestration (Spring Boot, MySQL, Redis).
- Integrated **GitHub Actions** for CI/CD, automating build, test, and container push processes.

Make My Study Plan — AI Powered Planner (Python & React) **2025**

- Engineered a Python-based recommendation engine to generate dynamic study schedules via REST APIs.
- Optimized application performance through efficient data parsing and modular microservices.
- Deployed on Vercel and AWS, managing environment variables and cloud-native configurations.

Leadership

Google Developer Group on Campus (GDGoC) **July 2025 – Present**
Lead — Web Development

- Overseeing technical stack decisions and mentoring students in Python, JavaScript, and Cloud fundamentals.
- Organizing HackNova Hackathon, coordinating infrastructure and technical deployment for 10+ teams.

Achievements

- **Winner - Build and Grow Hackathon (Jan 2026):** Won Pune regional for building EduGen, an agentic GenUI microservice platform using Python-based AI agents and Google Cloud.
- **1st Prize - Hacktoberfest (Oct 2025):** Developed an AI legal document analyzer using Python NLP libraries to extract risks and contractual obligations.
- **National Level Hackathons:** Participated in 7+ events, building complex tools like automated PDF parsers, SMTP email clients, and advanced web scrapers.

Experiment No: 2

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/crudDB")
.then(() => console.log("MongoDB Connected ✅"))
.catch(err => console.log(err));

const userSchema = new mongoose.Schema({
  name: String,
  email: String
});

const User = mongoose.model("User", userSchema);

// CREATE
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.send("User Created");
});

// READ
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// UPDATE
app.put("/users/:id", async (req, res) => {
  await User.findByIdAndUpdate(req.params.id, req.body);
  res.send("User Updated");
});

// DELETE
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
```

Experiment No: 1

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/mernDB")
.then(() => console.log("MongoDB Connected ✅"))
.catch(err => console.log(err));

app.get("/", (req,res)=>{
   res.send("MERN Environment Setup Successful");
});

app.listen(5000, ()=>{
   console.log("Server running on port 5000");
});
```

App.js

```
import { useEffect, useState } from "react";
import axios from "axios";
function App() {
 const [message, setMessage] = useState("Loading...");
 useEffect(() => {
  axios.get("http://localhost:5000/")
   .then(res => {
    console.log("Backend response:", res.data);
    setMessage(res.data);
   })
   .catch(err => {
    console.log("Error:", err);
    setMessage("Error connecting to backend");
 });
 }, []);
 return (
  <div style={{ textAlign: "center", marginTop: "100px" }}>
   <h1>{message}</h1>
  </div>
 );
}
export default App;
```

Output:

Server running on port 5000

MongoDB Connected ✅



